Module: ITNPBD3
Assessment: Relational and non-Relational Databases
Due Date/Time: 7th April 2026
AIAS Levels Allowed: 2

	Please tick the boxes/include information below as appropriate
Student ID Number	**3549063**
I have read and understand the severity of academic misconduct – see link below	✓
I give consent for my work to be used as an exemplar to future students.	✓
I have checked my submitted document to ensure it complies with module requirements.	✓
Link to OneDrive folder which contains evidence of the process I undertook to complete this assignment. *Please see notes below	**https://github.com/RedeemerNtumy/google-store-database**
I understand that failure to provide a working, and populated, OneDrive link means that my grade may be withheld – see link below – please tick the box.	✓
Tailored feedback. If you would like tailored feedback on a specific aspect (or aspects) of your work (e.g., code style, writing style, visualisation, discussion of results), then please give details here.	

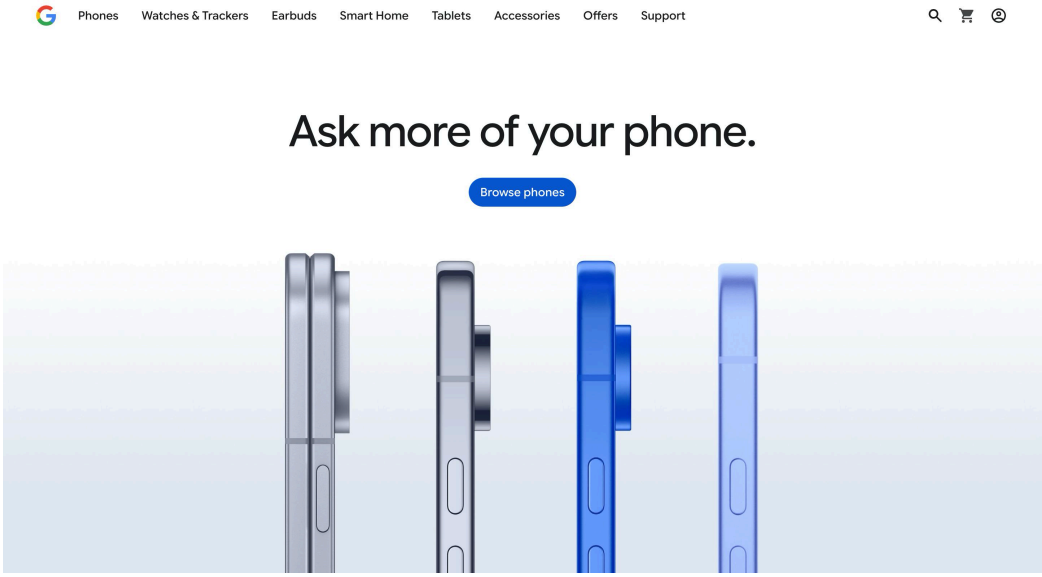
**This may include (but is not limited to) drafts, versions of the finished document, notes, references, AI output, and AI prompts. This will not be evaluated, nor graded but will provide assurance that the work submitted contains only the AI generated content in the final submission which is permitted in the AIAS scale for your assignment*

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/policy-and-procedure/

ITNPBD3 Assignment Spring 2026
3549063

Introduction

This report presents a database design analysis based on the Google Store (https://store.google.com/). The discussion covers both relational and non-relational approaches, highlighting how each model supports the functionality of a global online store.



Relational Database

1. Design Process

The relational design process begins with understanding the **business requirements** of the system. It sells hardware products (Pixel phones, Chromebooks, Nest smart-home devices, Fitbit wearables, etc). The database must support a product catalogue with variants, a user account and authentication system, a shopping cart and checkout flow, order management, and inventory tracking. The design process follows these steps:

- **Requirements Analysis:** I start by first identifying all data entities by walking through every user journey on the website. Each noun in these journeys is a candidate entity. These are **CUSTOMER**, **PRODUCT**, **ORDER**, **CART**, **CATEGORY**, and **INVENTORY**

- **ER Modelling:** I then map entities and their relationships. Based on my analysis of the website:
  - **A** CUSTOMER places **one** or **many** ORDERs, while **each** ORDER is associated with exactly **one** CUSTOMER.

  - **An** ORDER contains **one** or **many** PRODUCTs, and **a** PRODUCT can appear in **many** ORDERs. This many-to-many relationship can be later resolved using an associative entity, ORDER_ITEM, which stores attributes such as quantity and price.

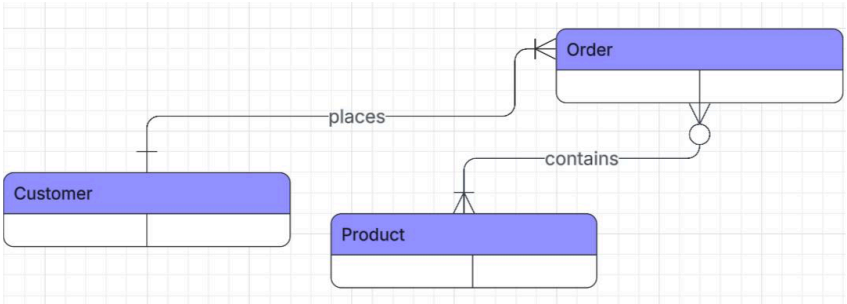
  - **A** CUSTOMER has **one** active CART.

  - **A** PRODUCT can appear in **many** CART_ITEMs across different carts.This births a bridge CART_ITEM to resolve the relationship between CART and PRODUCT while storing additional details like quantity.

- **A** CART contains **many** CART_ITEMs, and **each** CART_ITEM references a **single** PRODUCT.

- A PRODUCT may belong to **a** CATEGORY, where **one** CATEGORY contains **many** PRODUCTs, and each PRODUCT belongs to **one** CATEGORY.

This diagram below presents three chosen entities : CUSTOMER, ORDER, and PRODUCT, and their relationships . While additional entities would be required in a full system, only these three are included here to illustrate the fundamental structure of the database. This is using crow's foot notation based on the relationships I specified earlier



Info
A straight line (|) represents "one"
A crow's foot symbol represents "many"
A circle (O) indicates optional participation (zero)
Entities are represented as rectangles
Relationships are shown as labelled connectors between entities

- **Schema:** Ideally, the ER model will be converted into normalised relational tables up to Third Normal Form to reduce redundancy and ensure data consistency. Many-to-many relationships are resolved through associative entities including ORDER_ITEM and CART_ITEM. The PRODUCT entity is further refined by storing variants (such as colour and storage size) in a separate PRODUCT_VARIANT table. This supports granular stock management. For example, "Pixel 9 Pro / Hazel / 256GB" may be out of stock while "Pixel 9 Pro / Obsidian / 128GB" remains available. Both ORDER_ITEM and CART_ITEM reference PRODUCT_VARIANT_ID as a foreign key, separating product-level data from transactional data and improving overall data integrity.

CUSTOMER

- customer_id CHAR(36) PRIMARY KEY
- email VARCHAR(255) UNIQUE NOT NULL
- password_hash VARCHAR(255) NOT NULL
- name VARCHAR(255)
- phone VARCHAR(20)
- google_acct_id VARCHAR(255)
- created_at TIMESTAMP

ORDER

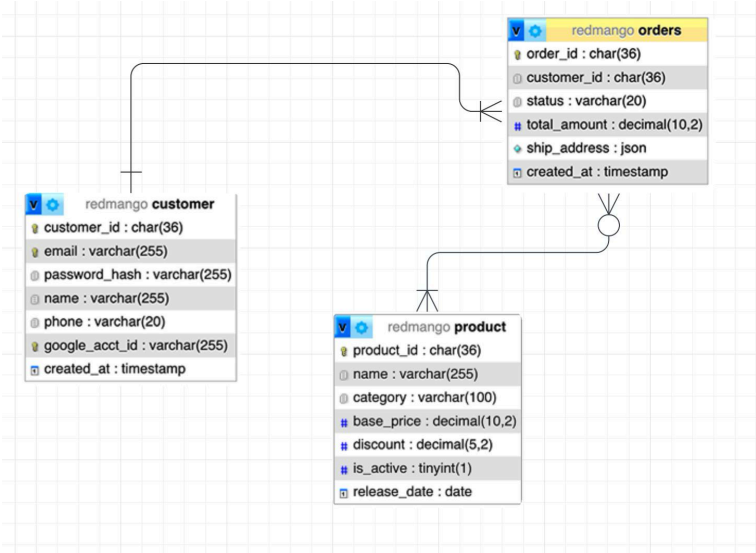
- order_id CHAR(36) PRIMARY KEY
- customer_id CHAR(36) NOT NULL
- status VARCHAR(20)
- total_amount DECIMAL(10,2)
- ship_address JSON
- created_at TIMESTAMP
- FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id)

PRODUCT

- product_id CHAR(36) PRIMARY KEY
- name VARCHAR(255)
- category VARCHAR(100)
- base_price DECIMAL(10,2)
- discount DECIMAL(5,2)
- is_active TINYINT(1)
- release_date DATE

- **Security Design:** Security cannot be forgotten. There is the need to encrypt sensitive columns like the user's personal details like email, house addresses, etc as well as hashing passwords at rest, and apply row-level security so customers can only query their own orders. There is the need to remember to parameterise queries to prevent SQL injection

Another thing worth noting is that the product_id on ORDER_ITEMS and the customer_id on ORDERS both need B-tree indexes for fast lookups during checkout.



2. Queries

The SQL queries below are written with the assumption that all supporting tables discussed earlier like bridge tables have already been fully implemented in the database schema. These queries rely on those existing relationships and structures, as would be expected in a complete production-level system similar to the Google Store, where all necessary entities and junction tables are already in place.

```sql
SELECT
  p.category,
  COUNT(DISTINCT o.order_id)        AS total_orders,
  SUM(oi.quantity * oi.unit_price) AS total_revenue,
  ROUND(AVG(oi.unit_price), 2)      AS avg_item_price
FROM orders o
JOIN order_item oi ON oi.order_id  = o.order_id
JOIN product    p  ON p.product_id = oi.product_id
WHERE o.status IN ('paid', 'shipped', 'delivered')
  AND o.created_at >= NOW() - INTERVAL '30 days'
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Query 1 : Revenue by Product Category
The Google Store merchandising team may need to see which product categories (Phones, Home, Wearables) generate the most revenue.

Query 2 : Customer Order History with Lifetime Value
They may also need to see all the orders being made and the amount by a particular customer. Lifetime value can also be extracted here as its an important metric for business teams. This query includes parameterisation to prevent SQL injection

```sql
SELECT
  o.order_id,
  o.created_at,
  o.status,
  o.total_amount,
  SUM(o.total_amount) OVER (
    PARTITION BY o.customer_id
    ORDER BY o.created_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS lifetime_value
FROM orders o
WHERE o.customer_id = $1
ORDER BY o.created_at DESC;
```